

# React

## 15-Page Concepts Guide for a Chatbot

A practical React reference focused on building and explaining a ChatGPT-style interface: components, props, state, hooks, forms, API calls, streaming, chat history, performance, routing, testing and deployment.

Best use: read this alongside your chatbot code and connect each concept to the component, state, API or backend flow where you use it.

# 1. React Fundamentals

## What is React?

- React is a JavaScript library for building user interfaces from reusable components.
- React is declarative: you describe what the UI should look like for the current state, and React updates the UI when state changes.
- A chatbot can be split into App, Sidebar, Chat, MessageList, Message, InputBox and LoadingIndicator components.

## Virtual DOM and rendering

- React keeps an in-memory representation of the UI and reconciles changes before updating the real DOM.
- The important idea is declarative UI and predictable reconciliation, not that React completely avoids DOM work.

## JSX

- JSX lets you write HTML-like syntax inside JavaScript. Use className instead of class and camelCase names such as onClick.

```
function Welcome() {  
  return <h1>Hello, chatbot!</h1>;  
}
```

## 2. Components & JSX

### Components

- Modern React primarily uses function components. A component receives inputs and returns UI.
- Keep components focused. Split a large chatbot component into message rendering, input, sidebar and API-related pieces.

### Conditional rendering

- Use &&, ternaries or early returns depending on readability.
- A chatbot can show loading, an empty state, errors or the message list based on state.

### Lists and keys

- Use `map()` to render message lists.
- Use a stable key such as a database message ID rather than an array index when messages can change.

### Example

```
{messages.map(message => (  
  <Message key={message._id} message={message} />  
))}
```

## 3. Props, State & Data Flow

### Props

- Props are read-only inputs passed from a parent to a child. A Chat component can pass a message object to Message.

### State

- State is changing data owned by a component that can trigger re-renders.
- Typical chatbot state: messages, input, loading, chatId, selected chat and error.

### One-way data flow

- React normally follows parent-to-child data flow. Parents can pass callbacks so children can request state changes.

### Updating state

- Never mutate state directly. Create a new value or use a functional updater.

```
setMessages(prev => [...prev, newMessage]);
```

## 4. useState Hook

### What it does

- useState adds local state to a function component and returns the current value plus a setter.
- Calling the setter schedules a re-render.

### Chatbot example

```
const [input, setInput] = useState('');
const [messages, setMessages] = useState([]);
const [loading, setLoading] = useState(false);
```

### Functional updates

- When the next state depends on the previous state, use `setState(prev => next)`.
- For arrays use `map`, `filter` and `spread` to create new arrays instead of mutating existing state.

### Common mistake

- A setter does not immediately change the current variable inside the same running function. The updated state is available on the next render.

## 5. useEffect & Component Lifecycle

### What useEffect is

- useEffect synchronizes a component with an external system such as an API, timer, browser event, subscription or WebSocket.
- It runs after React commits the render.

### Dependency array

- No array: after every render. Empty array: after initial mount. Dependencies: when a dependency changes.

### Chatbot example

```
useEffect(() => {  
  fetchChatHistory(chatId);  
}, [chatId]);
```

### Cleanup

- Effects can return cleanup functions for timers, listeners, subscriptions and aborting requests.

```
useEffect(() => {  
  const controller = new AbortController();  
  fetch(url, { signal: controller.signal });  
  return () => controller.abort();  
}, [url]);
```

### Common mistake

- Do not use useEffect for every piece of logic. Calculations and event-driven actions often belong directly in rendering or event handlers.

## 6. useRef, useMemo & useCallback

### useRef

- useRef stores a mutable value that persists across renders without causing a re-render when changed.
- For a chatbot it can point to the message container for scrolling to the latest message.

### useMemo

- useMemo memoizes a calculated value until dependencies change. Use it for genuinely expensive calculations or useful referential stability, not everywhere.

### useCallback

- useCallback memoizes a function reference. It can help when passing callbacks to memoized children.

### React.memo

- React.memo can skip rendering a component when its props have not changed. Use it when profiling shows a benefit.

### Interview distinction

- useRef = persistent mutable value. useMemo = memoized value. useCallback = memoized function. React.memo = memoized component rendering based on props.

## 7. Forms, Events & Chat Input

### Controlled components

- A controlled input gets its value from React state and updates it through onChange.
- This is useful for chatbot prompts because React always knows the current input.

### Example

```
const [input, setInput] = useState('');
<form onSubmit={handleSubmit}>
  <input value={input} onChange={e => setInput(e.target.value)} />
</form>
```

### Events

- React handlers use camelCase: onClick, onChange, onSubmit and onKeyDown.
- Use preventDefault() on a form event when you want to stop default browser submission.

### Chat UX

- Disable Send during a request if duplicate requests are not allowed.
- A common UX is Enter to send and Shift+Enter for a newline.

## 8. API Calls & Async Data

### fetch

- fetch returns a Promise. Check response.ok because fetch does not reject merely because the server returns 4xx or 5xx.

### Chat request

```
const response = await fetch('/api/promptPost', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ chatId, chat: input })
});
if (!response.ok) throw new Error('Request failed');
const data = await response.json();
```

### Request states

- Track loading, success and error states so the UI communicates what is happening.

### Race conditions

- If requests can overlap, cancel stale requests or associate each response with its originating request, especially when switching chats quickly.

### Axios

- Axios is an alternative HTTP client with features such as interceptors. It is optional because fetch is built into modern browsers.

## 9. Chatbot Architecture in React

### Suggested component tree

```
App
├── Sidebar
├── ChatList
├── ChatPage
├── MessageList
├── Message
└── ChatInput
```

### State ownership

- App can own selected chat if multiple areas need it. ChatPage can own messages, loading and input if those values are local to the page.
- Keep state as close as practical to the components that use it.

### Chat flow

- User types → ChatInput updates state → submit sends the API request → user message appears → backend generates response → assistant message renders → loading ends.

### Chat history

- When chatId changes, fetch its messages and replace the current message list. Keep chat metadata separate from message data when that makes updates easier.

# 10. Streaming Responses

## Why streaming?

- A chatbot feels faster when response text appears while the model is generating instead of waiting for the complete answer.

## Frontend idea

- The client reads a streaming HTTP response, decodes chunks and progressively updates the current assistant message.

## ReadableStream

```
const response = await fetch('/api/promptPost', options);
const reader = response.body.getReader();
const decoder = new TextDecoder();
let text = '';
while (true) {
  const { value, done } = await reader.read();
  if (done) break;
  text += decoder.decode(value, { stream: true });
  setAssistantMessage(text);
}
```

## Important

- A network chunk is not necessarily one token or complete message.
- Handle connection errors and cancellation.
- Avoid excessive state updates if chunks arrive extremely frequently.
- The backend must support streaming and the frontend must consume its response format.

# 11. Context API & Global State

## Context

- Context lets components access a value without manually passing it through every intermediate component.
- Good candidates include theme, authenticated user, language and app-level settings.

## Chatbot use

- A ChatContext could provide the current user, selected chat, chat list and actions if many distant components need them.

## Example

```
const ChatContext = createContext(null);

<ChatContext.Provider value={{ chats, selectedChatId }}>
  <App />
</ChatContext.Provider>
```

## Limitation

- Context is not automatically a complete state-management solution. Frequently changing context values can cause many consumers to re-render.
- For complex global state, Redux Toolkit, Zustand or another state library may be appropriate.

# 12. React Router & Navigation

## Why routing?

- Routing lets a single-page React app display different UI for different URLs without a full browser reload.

## Chatbot routes

- Possible routes: /login, /register, /chat, /chat/:chatId and /settings.

## Key concepts

- Routes map URL patterns to components. useParams reads dynamic parameters. useNavigate performs programmatic navigation. Link provides client-side navigation.

## Example

```
<Route path="/chat/:chatId" element={<ChatPage />} />
```

```
const { chatId } = useParams();
```

## Protected routes

- React can hide private pages when the user is not logged in, but real authorization must be enforced by the backend.

# 13. Performance & Re-rendering

## Why re-renders happen

- A component can re-render when its state changes, its parent renders, or consumed context changes.
- A re-render does not mean the entire real DOM is replaced. React reconciles the output.

## Chatbot performance

- Render long histories efficiently. Use stable keys. Keep frequently changing state local when possible. Consider virtualization for very large message lists.

## Common mistakes

- Using array indexes as keys for dynamic lists.
- Putting all application state into one huge context.
- Using useMemo/useCallback everywhere without evidence of a problem.
- Calling APIs during render instead of appropriate handlers/effects.

## Profiling

- Use React Developer Tools Profiler to identify expensive or unnecessary renders and optimize based on evidence.

# 14. Testing, Debugging & Production

## Testing

- Component tests check UI behavior. Integration tests check interactions among components and APIs. End-to-end tests verify complete user flows.
- Common tools include React Testing Library, Vitest, Jest and Playwright.

## What to test in a chatbot

- Message rendering.
- Prompt submission and API calls.
- Loading and error states.
- Chat history when chatId changes.
- Streaming updates.
- Protected navigation.

## Debugging

- React Developer Tools helps inspect components, props and state. Browser Network tools are essential for API debugging.

## Environment variables

- In Vite apps, client-exposed variables normally use the VITE\_ prefix. Never put database passwords or private API keys in frontend variables.

## Build and deployment

- `npm run build` creates a production build. The frontend can be deployed to a static hosting platform while the Node backend is deployed separately.

# 15. Interview Revision + Chatbot Mental Model

## Core questions

- 1. What is React and why is it used?
- 2. What is JSX?
- 3. Components and props?
- 4. Props vs state?
- 5. Explain useState.
- 6. Explain useEffect and dependencies.
- 7. What is useRef?
- 8. useMemo vs useCallback?
- 9. What is Context API?
- 10. Why are keys needed?
- 11. Controlled vs uncontrolled components?
- 12. How do you call an API?
- 13. How would you implement streaming?
- 14. How do you prevent unnecessary re-renders?
- 15. How do React Router and protected routes work?

## Chatbot mental model

- UI = components + JSX.
- Input = controlled state + events.
- Conversation = messages state.
- Selected conversation = chatId.
- Backend communication = fetch or Axios.
- Loading/errors = request state.
- Chat history = effect triggered by chatId.
- Streaming = ReadableStream + incremental state updates.
- Authentication = frontend manages user experience; backend verifies credentials/tokens.
- Performance = sensible state ownership, stable keys, profiling and selective memoization.

## Interview-ready explanation

• A React chatbot is a component-based UI where the conversation is represented by state. User input is controlled by React, and submitting the form sends the prompt to the backend. The backend returns the assistant response, which is added to message state and causes React to render the updated message list. For streaming, the frontend reads chunks from the response and incrementally updates the current assistant message. Chat history can be loaded whenever chatId changes, usually with useEffect. Authentication and authorization are enforced by the backend while React manages the user experience.

## Final rule

- Do not memorize hooks as isolated definitions. Understand what problem each hook solves and why it belongs in that part of your chatbot.